

Закони за мащабиране и нововъзникващи способности

Лекция 7

Част 2: Закони за мащабиране

Karlan, Chinchilla и как да разпределим изчислителния бюджет

Какво видяхме в тетрадката

В Част 1 обучихме 4–6 nanoGPT модела с различен размер върху един и същ корпус.

Наблюдения:

- По-големите модели имат по-ниска загуба (loss) — не е изненада
- На линейна скала кривата изглежда произволна
- На **лог-лог скала** се появи **права линия**

Правата линия на лог-лог скала = **степенен закон** (power law):

$$L(N) = aN^{-\alpha}$$

Това е *същият* модел, който изследователите от OpenAI намериха при мащаб **×1,000,000** по-голям от нашия.

Kaplan et al. 2020: три оси на мащабиране

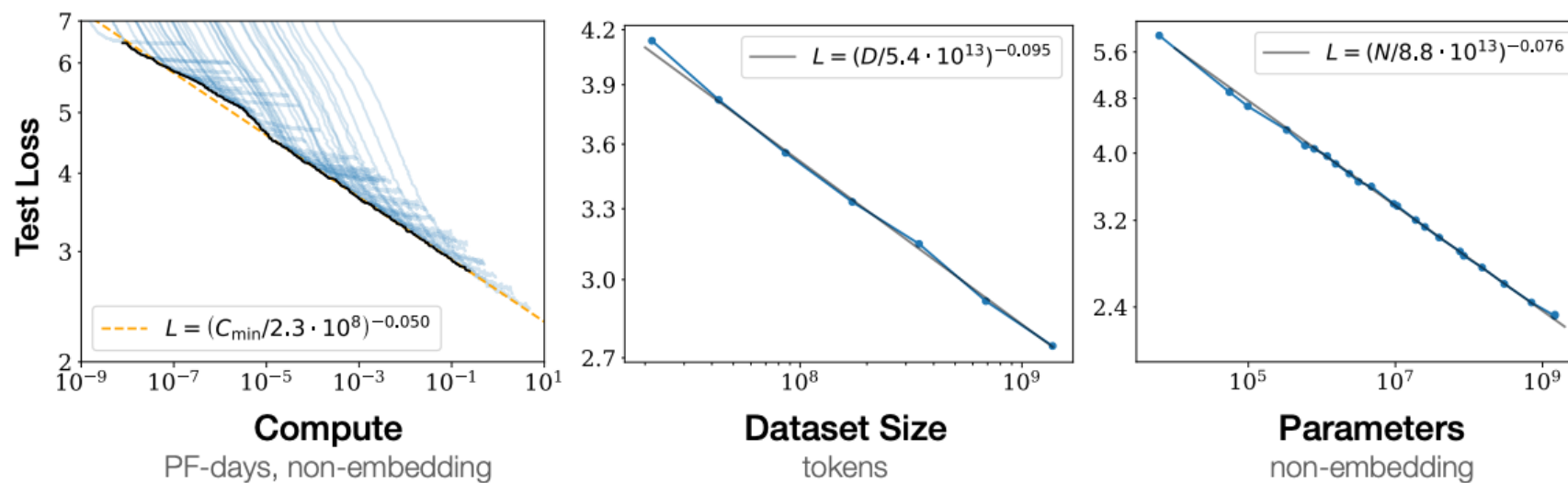
Статията на OpenAI измерва три оси поотделно:

Ос	Какво мащабираме	Закон
Параметри (N)	Брой тегла в модела	$L(N) \propto N^{-0.076}$
Данни (D)	Брой токени за обучение	$L(D) \propto D^{-0.095}$
Compute (C)	Общо изчисление (FLOPs)	$L(C) \propto C^{-0.050}$

Приблизителна връзка: $C \approx 6ND$ (FLOPs за обучение).

Ключов извод: подобренията са **плавни и предвидими** през много порядъци.

Kaplan et al.: три оси на лог-лог скала



Какво е предвидимо и какво — не

Предвидимо:

- Загубата при валидация (validation loss) — скалира плавно
- Перплексията — директно свързана със загубата

По-малко предвидимо:

- Представянето на конкретни задачи (downstream tasks)
- Точността на benchmark-ове — може да скочи рязко

Загубата е **агрегирана** мярка за качество. Конкретните задачи зависят от това *какво точно* е научил моделът — а това не е гарантирано от по-ниска загуба.

Тази разлика е централна за дебата за emergence по-нататък.

Оптимално разпределение: въпросът за милиони

Ако удвоим изчислителния бюджет, какво да направим?

Стратегия	Параметри	Данни
А) Удвои модела	$2 \times N$	$1 \times D$
В) Удвои данните	$1 \times N$	$2 \times D$
С) Раздели поравно	$\sqrt{2} \times N$	$\sqrt{2} \times D$

Kaplan (2020): модела трябва да расте **по-бързо** от данните.

→ Тренирай по-голям модел на сравнително по-малко данни.

Индустрията последва съвета. GPT-3: **175 милиарда параметъра**, обучен на ~300 милиарда токена.

Chinchilla (Hoffmann et al. 2022): корекцията

Основна находка: Karlan грешеше за оптималното съотношение.

Повечето модели бяха **масивно недообучени** — твърде много параметри, твърде малко данни.

Chinchilla правило: за оптимално обучение, токените трябва да мащабират **~1:1** с параметрите.

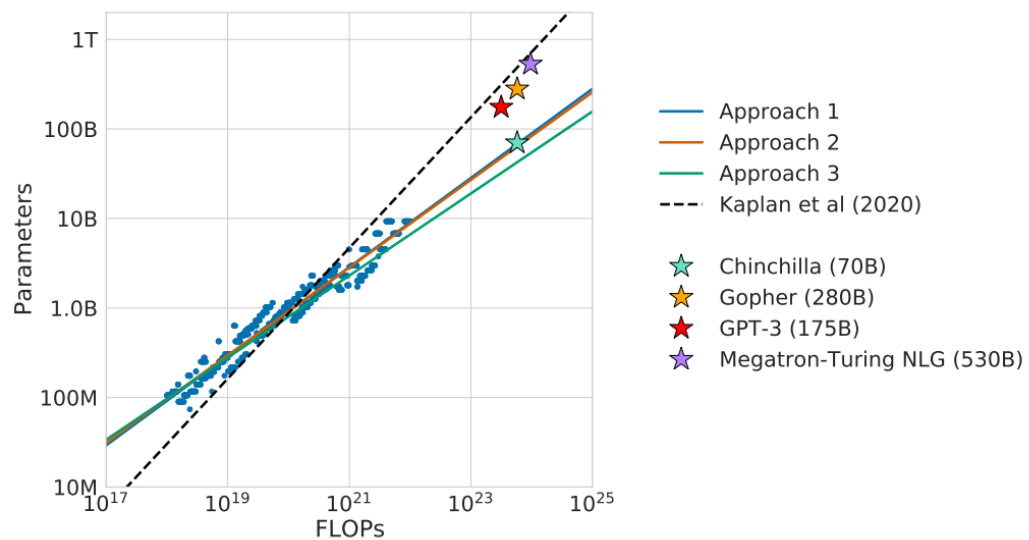
Практическо правило: **~20 токена на параметър**.

Chinchilla: конкретни числа

Модел	Параметри	Токени за обучение	Токени/параметър
GPT-3	175B	300B	1.7
Gopher	280B	300B	1.1
Chinchilla	70B	1.4T	20

Chinchilla (70B) постигна или надмина GPT-3 (175B) и Gopher (280B) — с **4x** по-малко параметри, но **4.7x** повече данни.

Chinchilla: оптимален compute бюджет



GPT-3 е далеч от оптималното съотношение параметри/данни.

Chinchilla промени индустрията

Преди Chinchilla (2020–2022): "направи модела по-голям"

След Chinchilla (2022+): "обучи по-малък модел на повече данни"

Модел	Параметри	Токени	Съотношение
GPT-3 (2020)	175B	300B	1.7
LLaMA 1 (2023)	65B	1.4T	21.5
LLaMA 2 (2023)	70B	2T	28.6
LLaMA 3 (2024)	70B	15T	214
Mistral (2023)	7B	>1T	>140

LLaMA следва Chinchilla. LLaMA 3 отива **далеч отвъд** — повече данни от оптималното, защото inference е по-евтин с по-малък модел.

Отвъд Chinchilla: inference-optimal training

Chinchilla оптимизира за **минимален compute при обучение**.

Но в практиката inference се извиква **милиони пъти** — по-малък модел е по-евтин.

Нова стратегия: обучи по-малък модел на **много повече** данни от Chinchilla оптималното.

$$\text{Общ cost} = \text{Training cost} + \text{Inference cost} \times \text{брой заявки}$$

Затова LLaMA 3 (70B) е обучен на 15T токена (214 на параметър) — далеч от 20:1, но inference е **2.5x по-евтин** от 175B модел.

Практически урок: как да мислим за бюджета

Имаме фиксиран GPU бюджет. Какво да направим?

Стъпка 1: Изчисли общия compute C (във FLOPs или GPU-часове)

Стъпка 2: Определи N^* и D^* :

- Chinchilla правило: $D^* \approx 20 \cdot N^*$
- За inference-heavy приложения: увеличи D^*/N^* допълнително

Стъпка 3: Провери дали имаш достатъчно данни за D^*

Почти винаги: по-малък модел на повече данни > голям модел на малко данни.

Benchmark-ове: как измерваме дали мащабирането работи

Benchmark-овете са стандартизирани тестове за измерване на способностите на моделите.

Benchmark	Година	Какво измерва	Колко задачи
GLUE	2018	Разбиране на език	9
SuperGLUE	2019	По-трудно разбиране	8
MMLU	2021	Знание по 57 теми	15,908
GSM8K	2021	Математическо разсъждение	8,500
HumanEval	2021	Генериране на код	164
BIG-Bench	2022	204 разнообразни задачи	~10,000

Наситяване на benchmark-ове

Benchmark-овете имат жизнен цикъл — създават се, модели ги решават, стават безполезни.

Benchmark	Създаден	Наситен	Време до наситяване
ImageNet	2010	~2017	7 години
GLUE	2018	~2020	2 години
SuperGLUE	2019	~2020	~1 година
MMLU	2021	~2024	~3 години
BIG-Bench Hard	2022	~2024	~2 години

Темпото се ускорява: това, което отнемаше десетилетие, сега отнема 2–3 години.

Когато benchmark е наситен, той **вече не измерва способност** — трябва нов benchmark-ове постоянно.

Закон на Goodhart и замърсяване

"Когато една мярка стане цел, тя престава да бъде добра мярка."

Проблеми с benchmark-овете:

1. **Оптимизация за benchmark:** лаборатории оптимизират директно за конкретни тестове
2. **Замърсяване** (от Лекция 6): обучаващите данни могат да съдържат тестовите въпроси
3. **Затворени модели:** не можем да проверим за замърсяване при GPT-4, Claude, Gemini

Практическо следствие: публикуваните сравнения между модели трябва да се четат **скептично**.

Benchmark резултатите са необходими, но не достатъчни.

Преход: от загуба към способности

Всичко досега беше за **загубата** — непрекъснатата метрика, която скалира плавно.

Но GPT-3 (Brown et al. 2020) показва нещо **качествено различно**:

При достатъчен мащаб моделите започнаха да правят неща, които малките модели **просто не можеха**.

Нека видим какви бяха тези неща — и да тестваме едно от тях сами.

Част 3: Какво позволява мащабът

In-context learning и нововъзникващи способности

GPT-3: нова парадигма

Brown et al. (2020) показаха, че достатъчно голям модел може да решава задачи **без фина настройка** — само от примери в подканата (prompt).

Три режима:

Режим	Какво подаваме	Пример
Zero-shot	Само описание на задачата	"Преведи на английски: Котката спи"
One-shot	Едно решено задание + новото	"Куче→Dog. Котка→"
Few-shot	2–64 решени задания + новото	"Куче→Dog. Къща→House. Котка→"

Без промяна на теглата. Без обучение. Само **текст в контекста**.

In-context learning: как работи

Какво НЕ е in-context learning:

- Не обновява теглата на модела
- Моделът не "учи" в смисъла на градиентно спускане
- Не е фина настройка

Какво Е:

- Моделът използва примерите в подканата като **имплицитна супервизия**
- Задачата се разпознава чрез **съпоставяне на модели** (pattern matching) от контекста
- Моделът е видял подобни формати по време на предварителното обучение

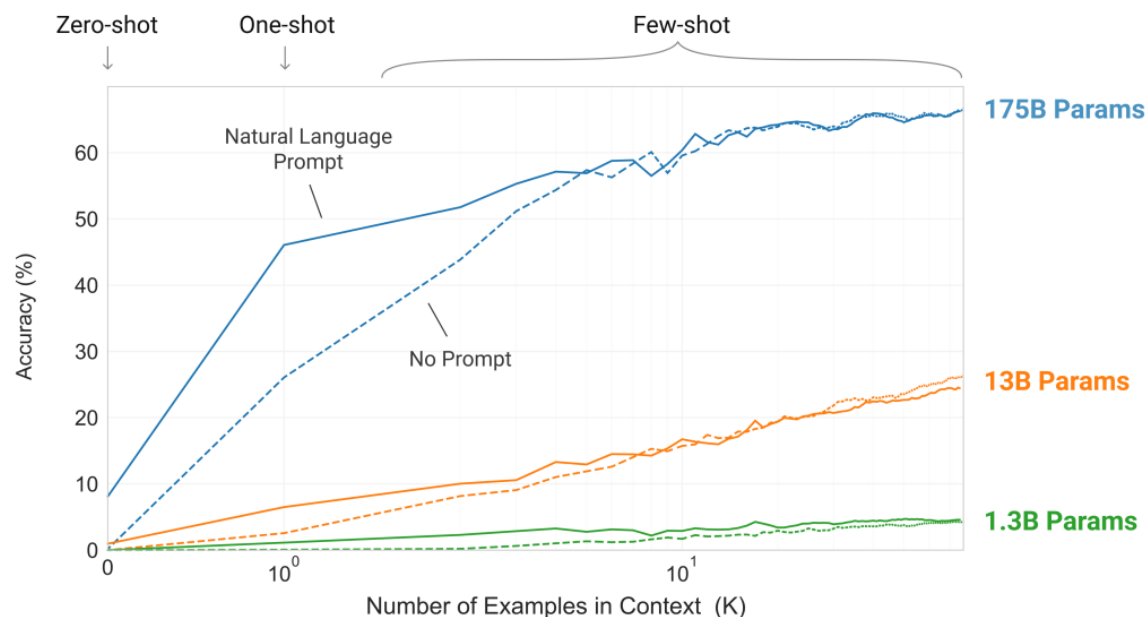
Защо работи само при мащаб:

- Моделът трябва да има достатъчен **капацитет** да представи разнообразни формати
- И да се **адаптира** в рамките на един forward pass

In-context learning скалира с размера на модела

Ключово наблюдение от GPT-3:

- Малките модели почти **не се подобряват** от примери в подканата
- Големите модели се подобряват **значително**



GPT-2 (124M) — нашият модел — е на **долната граница** на това да работи.

Пример: аритметика с in-context learning

Zero-shot:

Q: What is $45 + 67$?
A:

GPT-2: почти винаги грешен отговор.

Few-shot:

Q: What is $12 + 8$?
A: 20

Q: What is $34 + 21$?
A: 55

Q: What is $45 + 67$?
A:

GPT-2: понякога познава, често греша. GPT-3 175B: **правилен в >95% от случаите** за двуцифрени числа.

Нека видим на практика с GPT-2 → тетрадка

Какво видяхме в демото

GPT-2 (124M) е **непоследователен** при in-context learning:

- Простите задачи понякога успяват
- По-сложните почти винаги се провалят
- Резултатите варират от опит на опит

Сравнение с публикувани GPT-3 резултати на същите типове задачи:

Задача	GPT-2 (124M)	GPT-3 (175B)
Превод на думи	~30%	~80%
Аритметика (2 цифри)	~20%	~95%
Класификация на настроения	~50%	~85%

Разликата не е само **количествена** — тя е **качествена** за тези задачи.

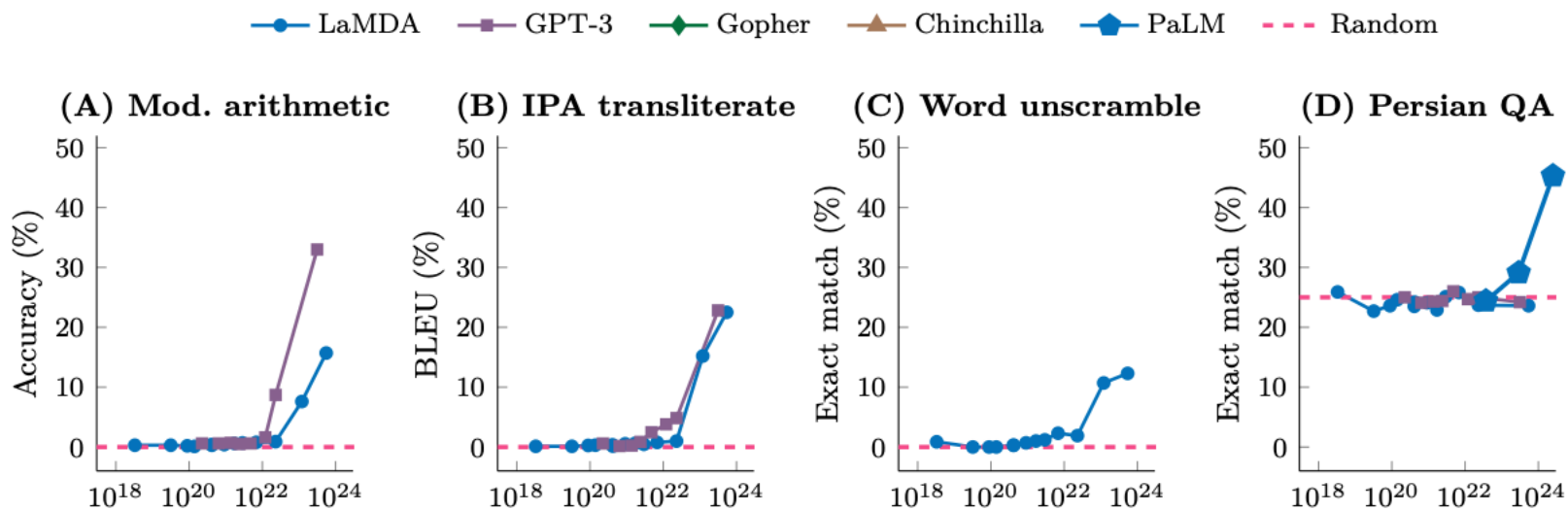
Нововъзникващи способности (Emergence)

Когато повече $\neq$ просто по-добро

Wei et al. 2022: emergence

Дефиниция: една способност е *НОВОВЪЗНИКВАЩА*, ако:

1. Не присъства при по-малки модели
2. Появява се при по-големи модели
3. Преходът е **рязък**, а не постепенен



Примери за emergence

Способност	Под прага	Над прага
Аритметика с много цифри	Случаен отговор	Правилно решаване
Разбъркване на букви (word unscrambling)	Не разбира задачата	Решава я
Многоезиково разсъждение	Превежда, но не разсъждава	И двете заедно
Мисловни вериги (chain-of-thought)	Произвежда безсмислени стъпки	Коректни стъпки

Прагът обикновено е около **10В–100В параметъра** — зависи от задачата и данните.

Защо emergence е важно

Ако способностите са наистина нововъзникващи:

1. **Не можем да ги предвидим** от малки модели — трябва да обучим големия, за да разберем
2. **Мащабирането има изненади** — не знаем предварително какво ще може моделът
3. **Планирането е трудно** — не можем напълно да определим способностите преди обучение

Това е фундаментално различно от плавното мащабиране на загубата (Kaplan).

Загубата скалира плавно. Способностите — може би не.

Schaeffer et al. 2023: emergence като мираж?

Основен аргумент: рязкият преход е артефакт на **избора на метрика**, а не свойство на модела.

Примерът с аритметиката:

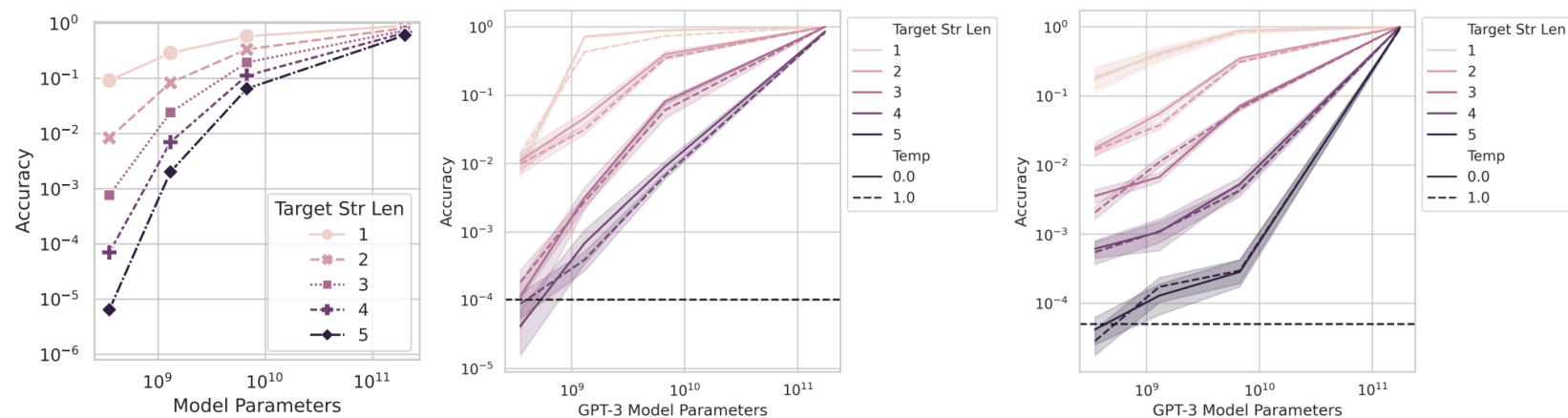
Метрика	Какво измерва	Какво виждаме
Exact match	Целият отговор е правилен	Рязък преход: 0% → 90%
Частичен кредит (per digit)	Колко цифри са правилни	Плавно подобрене

За да отговориш "1234" правилно с exact match, трябва **всичките 4 цифри**.

Ако моделът прогресира от 1 вярна → 2 → 3 → 4 цифри, exact match показва 0, 0, 0, 1 — **скок**.

Но подобренieto е **плавно**, маскирано от нелинейната метрика.

Визуална илюстрация на аргумента



При линейна метрика "emergence" изчезва — плавно подобрение вместо рязък скок.

Какво означава критиката

Schaeffer НЕ казва, че мащабирането не помага.

Schaeffer казва, че *как измерваме* определя *какво виждаме*.

Твърдение	Статус
По-големите модели са по-добри	Безспорно
In-context learning скалира с размера	Безспорно
Преходът от "не може" до "може" е рязък	Зависи от метриката
Emergence е фундаментално свойство	Спорно — може би е артефакт

Връзка с наситяването на benchmark-ове: **изборът на метрика оформя това, което виждаме.**

Честен обзор: кой е прав?

И двете страни имат основание:

Wei et al. (emergence е реално):

- Има задачи, които малки модели наистина *не разбират*
- In-context learning не е просто "по-точни отговори" — то е *качествено нова способност*
- Мисловните вериги (chain-of-thought) не работят при малки модели

Schaeffer et al. (emergence е мираж):

- Много от "рязките преходи" изчезват при смяна на метриката
- Методологията на измерване е критична
- Не всяко apparent emergence е реално emergence

Практически извод: тълкувайте benchmark резултати внимателно, особено близо до прагови региони.

Какво НЕ е спорно

Независимо от дебата за emergence, няколко неща са установени:

- По-големите модели са **значително по-добри** в in-context learning
- **Instruction following** (следване на инструкции) се подобрява драматично с мащаба
- **Разсъждението** (reasoning) скалира — макар и степента да е дискуссионна
- **Мултиезикови способности** се подобряват (включително за български)

Механизмът и точната форма на кривата са дискусийни.

Реалните способности — не.

Обобщение

Тема	Ключов резултат	Източник
Параметри vs загуба	Степенен закон: удвояване на параметрите → предвидим спад	Kaplan et al. 2020
Оптимално обучение	~20 токена на параметър; тренирай по-малък модел на повече данни	Chinchilla 2022
In-context learning	Few-shot скалира със размера на модела	GPT-3 2020
Emergence	Рязки прагове на способности при мащаб	Wei et al. 2022
Критика на emergence	Рязките преходи може да са артефакти на метриката	Schaeffer et al. 2023
Наситяване на benchmark-ове	Benchmark-ове се наситяват за 2–3 години; оценяването е трудно	—

Ключови изводи

1. **Загубата скалира предвидимо** — степенни закони през много порядъци
2. **Chinchilla**: обучи по-малък модел на повече данни — не обратното
3. **In-context learning** е способност, която се появява при мащаб и е основата на промптинг техниките (Лекция 10)
4. **Emergence** вероятно е комбинация от реални прагове и метрични артефакти
5. **Benchmark-овете не са истината** — те са приближение, което се износва бързо

Тийзър за Лекция 8

Сега имаме мощни базови модели:

- Обучени на масивни данни
- С предвидимо поведение на мащабиране
- С in-context learning и други способности

Но суровият предварително обучен модел **предвижда токени** — той не е асистент, не следва инструкции и може да произведе вредно съдържание.

Лекция 8: Как превръщаме езиков модел в нещо **полезно и безопасно**?

→ Съгласуване (alignment), обучение с инструкции (instruction tuning), RLHF

Дискусия

- Ако загубата скалира предвидимо, защо резултатите на конкретни задачи са по-трудни за предвиждане?
- Chinchilla казва "тренирай по-малък модел на повече данни" — защо лабораториите все пак обучават модели с 70B+ параметъра?
- Ако GPT-2 демото за few-shot в голяма степен се провали — при колко параметъра мислите, че ще заработи надеждно?
- Критиката на Schaeffer означава ли, че emergence не е реално? Как би изглеждал убедителен тест?

Ресурси

- Kaplan et al. 2020 — *Scaling Laws for Neural Language Models*
- Hoffmann et al. 2022 — *Training Compute-Optimal Large Language Models* (Chinchilla)
- Brown et al. 2020 — *Language Models are Few-Shot Learners* (GPT-3)
- Wei et al. 2022 — *Emergent Abilities of Large Language Models*
- Schaeffer et al. 2023 — *Are Emergent Abilities of Large Language Models a Mirage?*
- Ganguli et al. 2022 — *Predictability and Surprise in Large Generative Models*